

# Kapitel 20. Exceptions

## Innehållsförteckning

[Vad är \*exceptions\*](#)

[Använda exceptions](#)

[Ofångade exceptions](#)

[Hierarkier med exceptions](#)

Detta kapitel behandlar det avancerade felhanteringssystemet i C++ som kallas *exceptions* (undantag). Exceptions tillåter mycket flexibel hantering och rapportering av fel.

## Vad är *exceptions*

Ett programs robusthet mäts bra på hur väl det hanterar och återhämtar sig från olika felsituationer. I ett idealiskt program skall ett fel hanteras på korrekt sätt och sedan skall exekveringen fortsätta så gott det går. Det är dock ganska svårt att i ett stort program förmedla felsituationer mellan olika delar av programmet. Traditionellt i C och många andra språk har man förmedlat felmeddelanden i ett program antingen genom vissa överenskomna returnvärden från funktioner eller genom att helt enkelt avsluta programmet. Den första modellen har sina fördelar och nackdelar. I följande funktion fungerar denna helt ok:

```
Image * createImage (string Filename) {
    // försök läsa filen
    if ( ! readFile ( Filename ) {
        // kunde inte läsa filem
        return 0;
    }

    // returnera en ny Image
    return new Image ( ... );
}

int main () {
    string File = ....;

    // försök läsa en fil
    if ( ( NewImage = createImage (File ) ) == 0 ) {
        // kunde inte läsa filen, hantera felet
        ....
    }
    ...
}
```

Programmet ovan försöker läsa en fil och skapa en `Image` på bas av läst data. Om filen inte kunde läsas returnerar funktionen värdet 0. Detta värde är enkelt att jämföra med, eftersom 0 aldrig är en adress till ett giltigt objekt. Vi har i detta fall ett bra returvärde som indikerar fel. I följande exempel är det inte lika enkelt längre:

```
float divide (float X, float Y) {
    // dividerar vi med 0?
    if ( Y == 0 ) {
```

```
// jep, vad ska vi göra nu? Vilket värde skall returneras?  
cout << "Division med 0" << endl;  
}  
else {  
    return X / Y;  
}  
}
```

Funktionen `divide()` får problem om användaren försöker dividera med 0, vilket inte är tillåtet. Funktionen kan skriva ut ett felmeddelande på skärmen, men det kanske inte ses av användaren, eller så kan den terminera programmet, vilket inte heller är en bra ide. Det finns inte heller något returvärde som kunde indikera att ett fel har skett. Problemet blir svårare om ett fel hnder djupt nästlat i funktionsanropen, och koden som kan hantera felet är flera nivåer uppåt i anropshierarkin. Programmet måste då på något meningsfullt sätt förmedla information om felet till felhanteraren.

I sammanhang som dess är C++ *exceptions* väldigt praktiska. De är C++:s sätt att förmedla olika typer av fel. Med exceptions slipper man bry sig om dylika problem och har större frihet att utnyttja t.ex. returvärdet till "nyttig" information och inte felmeddelanden. Exceptions kan även förmedla mera information, såsom t.ex. en sträng som innehåller en text som kan skrivas ut på skärmen samt annan information om exakt vad som gick fel.

---

[Förutgående](#)  
Stänga en fil

[Hem](#)

[Nästa](#)  
Använda exceptions